

A Survey on Prompting Techniques in LLMs

Prabin Bhandari

Department of Computer Science

George Mason University

Fairfax, Virginia, USA

pbhanda2@gmu.edu

Abstract—Autoregressive Large Language Models have transformed the landscape of Natural Language Processing. Pre-train and prompt paradigm has replaced the conventional approach of pre-training and fine-tuning for many downstream NLP tasks. This shift has been possible largely due to LLMs and innovative prompting techniques. LLMs have shown great promise for a variety of downstream tasks owing to their vast parameters and huge datasets that they are pre-trained on. However, in order to fully realize their potential, their outputs must be guided towards the desired outcomes. Prompting, in which a specific input or instruction is provided to guide the LLMs toward the intended output, has become a tool for achieving this goal. In this paper, we discuss the various prompting techniques that have been applied to fully harness the power of LLMs. We present a taxonomy of existing literature on prompting techniques and provide a concise survey based on this taxonomy. Further, we identify some open problems in the realm of prompting in autoregressive LLMs which could serve as a direction for future research.

Index Terms—Natural language processing, language models, autoregressive large language models, prompting

I. INTRODUCTION

Language models (LMs) have long been the de facto standard for modeling natural languages, designed with the purpose of comprehending and generating human-like language. Language models also serve as the cornerstone upon which a multitude of Natural Language Processing (NLP) applications are built, including but not limited to machine translation, sentiment analysis, document classification, and chatbots. This has made LMs one of the most extensively researched domains within the field of NLP. Initially, language modeling relied on statistical approaches, which eventually gave way to neural language models fueled by the increase in our computing resources. With neural LMs came the requirement for substantial labeled data, as any downstream task was accomplished via supervised training. The introduction of Transformers [1] marked a pivotal shift, introducing the pre-train and fine-tune paradigm of NLP. Transformers gave rise to pre-trained language models (PLMs). These PLMs, while still being neural LMs, mostly used architecture similar to the transformer or a variant of it. The key difference between neural LMs and PLMs is in their training process. PLMs are pre-trained on a vast amount of textual data in a semi-supervised approach, eliminating the need for labeled data during pre-training. Following their pre-training, PLMs are fine-tuned for specific downstream tasks using relatively less labeled data than what neural LMs typically require. This is because PLMs have a strong foundation in language

modeling through pre-training. Recently, the pre-train and fine-tune paradigm has evolved into a pre-train and prompt approach, mainly due to the emergence of Large Language Models (LLMs).

LLMs evolved from PLMs as researchers tried to enhance the performance of PLMs by increasing the model’s size, dataset volume, and/or computational resources employed during training. This drive was motivated by scaling law [2]. In doing so, the performance of these models did indeed increase as expected. However, it also led to the rise of emergent abilities [3] that cannot be explained solely by scaling law. Emergent abilities refer to the capabilities of an LLM that manifest when it reaches a certain scale, achieved through an increase in the number of parameters, pre-training data, computational resources, or a combination of these factors. The majority of LLMs nowadays are autoregressive in nature. This means that they generate the next token based on the previous tokens only. These autoregressive LLMs use either the encoder-decoder architecture or only the decoder of the transformer. Given that these LLMs have been pre-trained on a plethora of text and often the whole internet, they are an extremely powerful tool. However, harnessing their immense potential could be challenging due to their large size, making it infeasible to fine-tune them for each downstream task. This challenge has been mitigated by the emergence of prompting techniques in LLMs, which seek to replace the need for fine-tuning LLMs for each downstream task individually.

Prompting refers to providing specific instruction or input, whether in human-readable form or not, to the LM in order to accomplish the downstream task. The use of prompting first appeared in the context of PLMs. For instance, Petroni et al. [4] used prompting when attempting to quantify the amount of world knowledge embedded in PLMs. Their hypothesis was that since PLMs were pre-trained on vast amounts of data, PLMs might contain significant world knowledge embedded within their parameters. Concretely, their input to the PLM was: *Francesco Bartolomeo Conti was born in [MASK]*.. Since they employed a masked language model in their experiments, the expectation was that the model would replace the *[MASK]* token with the correct answer. Prompting with respect to autoregressive LLMs was popularized by Brown et al. [5]. They demonstrated an emergent ability of LLMs, facilitated by prompting techniques, known as in-context learning. Basically, their research shows that LLMs, when provided with a task instruction and a few demonstration examples, could handle a

wide range of downstream NLP tasks. This novel prompting technique for LLMs eliminated the need to fine-tune an LLM and opened a new field of research into prompting techniques for LLMs.

Given the recent proliferation in prompting techniques, this paper discusses the current literature concerning prompting techniques tailored for autoregressive LLMs. The structure of the paper is as follows. Section II lays the groundwork by discussing the preliminaries required to follow the rest of the paper. Section III introduces the taxonomy used to categorize prompting techniques in autoregressive LLMs. Section IV provides a concise survey of existing literature, organized according to the established taxonomy. Section V discusses the prevailing challenges and open problems in the field of prompting, offering potential future research directions. Finally, we conclude with our remarks in Section VI.

II. PRELIMINARIES

A. Language Models

A language model (LM) is a computational model of a natural language that predicts and generates human language based on a text corpora that it was trained on. Technically, LMs model the likelihood of a sequence of tokens in order to predict the probabilities for the missing or next token. Language models can broadly be classified into four main development stages:

1) *Statistical language models (SLM)*: SLMs, introduced in the early 1980s, are based on statistical learning methods. They assign probabilities to tokens based on a statistical model of a text corpora. One of the notable SLMs is called the N-gram model. The N-gram models are based on Markov's assumption that the probabilities of the next token are based on the last $(N - 1)$ tokens. Formally,

$$P(w_n|w_{n-1:1}) = P(w_{n-1}|w_{n-2:1}) * P(w_{n-2}|w_{n-3:1}) * \dots * P(w_2|w_1) * p(w_1) \quad (1)$$

The probabilities are estimated using a Maximum Likelihood Estimate (MLE) based on the frequency of the tokens in the text corpus. Along with n-gram models, the Hidden Markov Model (HMM) based language models, and maximum entropy-based language models were some of the other popular SLMs.

2) *Neural language models (NLM)*: NLMs leverage neural networks to estimate token probabilities. While various neural network architectures, such as feed-forward neural networks (FFNN) and convolutional neural networks (CNN), have been explored for language modeling, recurrent neural networks (RNN) emerged as the dominant choice. However, the landscape underwent a significant transformation with the introduction of Transformers [1].

Transformer, originally purposed for machine translation tasks, features an encoder-decoder architecture. In this design, the encoder encodes the input token sequence to extract context, while the decoder's main role is to generate the next token based on the encoder's encoded information and already decoded tokens. Transformers introduced a technique called

self-attention which captures complex contextual relationships within sequences. The original transformer used multi-headed self-attention, enabling the model to attend to different parts of the input sequence simultaneously, significantly boosting its capacity to model complex dependencies.

3) *Pre-trained language models (PLM)*: Earlier efforts for PLMs involved training shallow networks for word embeddings, like word2vec [6] and Glove [7], to capture the semantic meaning of the word. Although these models were highly effective, they failed to capture essential contextual information related to a word. Subsequent models like ELMo [8], employed RNNs in an effort to gather contextual information in the word embeddings; however, they were constrained by their size.

With the introduction of transformers, researchers were able to train more deeper networks. Further, the advent of self-supervised learning techniques [9] meant that costly human-annotated data are not required to train the LMs. In the self-supervised learning paradigm, the LMs are pre-trained with the help of raw textual data only, completely bypassing the need for human-annotated data. These PLMs are often then fine-tuned for specific downstream tasks with human-annotated data. However, due to the effectiveness of pre-training, the requirement for the size of human-annotated data drops off.

Various pre-training objectives are used with alterations to the standard encoder-decoder architecture which gives rise to three different types of PLMs: (a) Left-to-right PLM, (b) Masked PLM, and (c) Encoder-decoder PLM. Figure 1, extracted from [10], illustrates the architectures of these three different PLMs. Further explanation of these PLMs is provided below:

a) *Left-to-Right PLM*: Left-to-right (or similarly Right-to-left) PLMs, like GPT [11] and its successor GPT-2 [12], are autoregressive language models that usually employ the decoder part of the original transformer architecture and are pre-trained to predict the upcoming tokens one at a time based on the previous tokens using a large text corpus.

b) *Masked PLM*: Masked PLMs are non-autoregressive in nature meaning that they consider both the preceding and following tokens when predicting token probabilities. Masked PLMs generally use the encoder part of the original encoder-decoder architecture of the transformer. These PLMs are pre-trained by applying a noising function that corrupts the text and the pre-training objective is to restore the original uncorrupted text. An early example of such a PLM is BERT [13], where certain tokens within a sentence are replaced with *[MASK]* and the pre-training objective was to predict the *[MASK]* token. Other various noising functions have also been proposed [14–16].

c) *Encoder-Decoder PLM*: Encoder-Decoder PLMs use the original architecture of the transformer. While they are generally auto-regressive in nature, there are a few models purposed with non-autoregressive design. Typically, a noising function is applied to the encoder, while a standard language modeling is used at the decoder part during the model's pre-

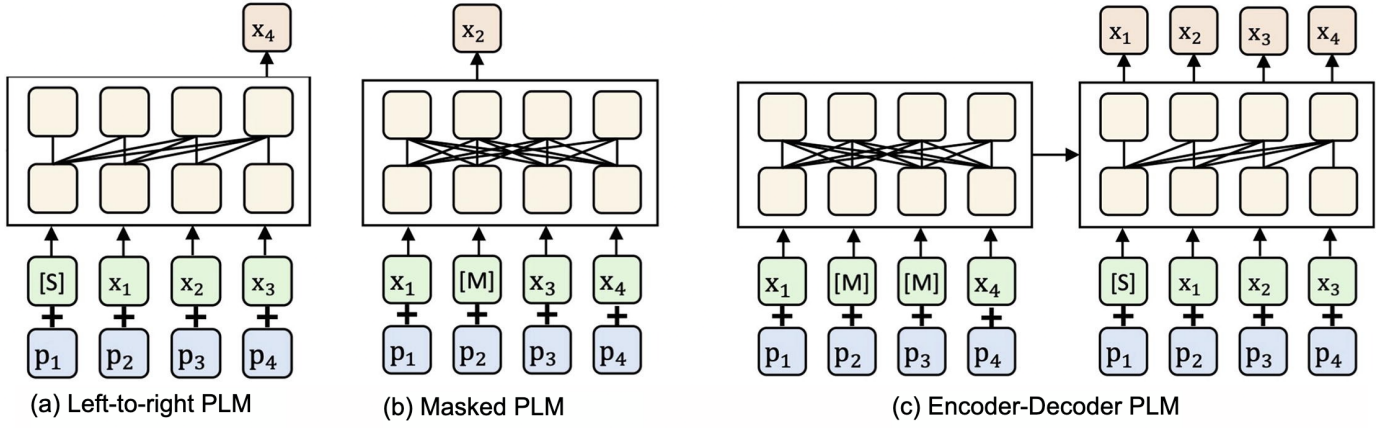


Fig. 1. Architecture of different PLMs. Image from [10].

training. Notable examples of Encoder-Decoder PLMs include Meta’s BART [17] and Google’s T5 [18].

4) *Large language models (LLM)*: The scaling law [2] dictates that an increase in the model’s size, dataset size, and computational resources used during training often results in enhanced performance on the downstream tasks. Researchers have tried constantly to push the boundaries of this law by continually increasing the model’s size. For instance, GPT-3 [5] has a massive 175 billion parameters, while PaLM [19] surpasses even that with 540 billion parameters. Despite having similar training methodologies compared to other PLMs, these large PLMs exhibit *emergent abilities* [3]. For example, GPT-3 can learn a task description with the help of a few examples passed as context whereas the predecessor of GPT-3, GPT-2 can not do that. In contemporary times, the term “Large language models (LLMs)” primarily refers to these massive language models, having tens or even hundreds of billions of parameters, and trained on vast datasets. These LLMs predominantly adopt Left-to-right transformer architecture (decoder-only), and they commonly exhibit an autoregressive nature.

B. Prompting

Prompting refers to providing a specific input or instruction to guide the model’s output. Basically, an input x with the help of a prompt template f is converted to a new representation $f(x)$, which is then fed into the model to get the desired output y . We generally employ two kinds of prompts, cloze prompts, and prefix prompts.

Cloze prompts are popular with masked language models, where the objective is to fill in the blanks. For example, if our task is to find the capital city of a country, our cloze-style prompt will be as follows:

The capital of Nepal is [BLANK].

The model is expected to fill the blank with the correct answer.

Prefix prompts are generally employed with autoregressive LLMs, where the goal is for the model to produce the continuation of the string input. For example, if our task is

to convert a sentence in English to a sentence in Nepali, our prefix-style prompt will be as follows:

Convert the following English sentences to Nepali:
 English: All the world’s a stage, and all the men and women merely players.
 Nepali:

The model is expected to produce a continuation of this input where the output will be the Nepali translation of the input English sentence.

If we provide these templates without additional examples, it is referred to as zero-shot prompting. However, if we provide a few illustrative examples of the correct inputs and outputs, it is referred to as few-shot prompting.

III. AREA TAXONOMY

Figure 2 presents the area taxonomy of prompting methods in autoregressive LLMs. The classification of prompts is based on two key dimensions: the level of human involvement in prompt creation and the specific types of these prompts. In terms of human effort, prompts are categorized into two groups: “Hand-Crafted” and “Automated”, reflecting the extent of manual input required in the prompt creation process. Additionally, prompts are categorized into three distinct groups based on their intended objectives. These categories include “Task-Based”, “Generate-Auxiliary” and, “Resource/Tools Augmented”. It is important to note that this classification is based on the goals and purpose of the prompts themselves, rather than the ultimate objective of the downstream tasks. In the next section, we delve into existing research within each of these classifications to provide a comprehensive overview of the field.

IV. TAXONOMY-BASED SURVEY

In this section, we offer a survey of the existing literature about prompting within the domain of autoregressive LLMs, structured according to the taxonomy introduced in section 3.

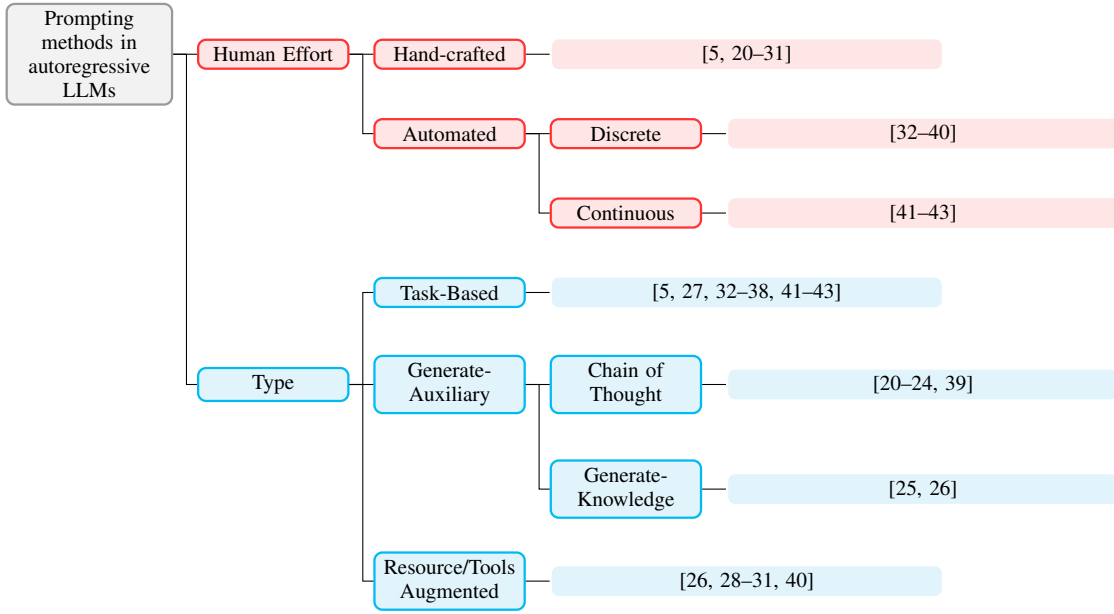


Fig. 2. Area Taxonomy of prompting methods in autoregressive LLMs.

It is noteworthy that while some of the research discussed may not be exclusive to autoregressive LLMs, or may have originally targeted PLMs, many of these approaches are applicable and adaptable for effective use with autoregressive LLMs.

A. Human Effort

On the basis of the amount of human effort required to create the prompts, they can be classified into Hand-crafted and Automated.

1) *Hand-crafted Prompts*: Hand-crafted prompts are the most natural way of creating prompts where a prompt template is created based on human intuition. Brown et al. [5] introduced hand-crafted prefix prompts for solving a variety of tasks. We provide an example of a hand-crafted prompt taken from [5]:

```
Translate English to French:
Cheese =>
```

The prompt above is called a zero-shot prompt as we have only provided the task description along with the input. If we provide a few input-output examples along with the task description, we call them few-shot prompts. An example of a few-shot prompt is provided below:

```
Translate English to French:
Sea otter => loutre de mer
plush girafe => girafe peluche
Cheese =>
```

2) *Automated Prompts*: Although manual prompt creation is intuitive, such hand-created prompts may have limitations. One such limitation is that these hand-crafted prompts may be sub-optimal. Another issue is that hand-crafted prompts are domain-specific and it can be an arduous task to hand-craft

a prompt for some complex downstream tasks. Consequently, researchers are exploring automated methods for prompt template design. These automatically generated prompts can be further classified into discrete and continuous prompts.

a) *Discrete Prompts*: Discrete Prompts, also referred to as “hard prompts”, are those prompts where the prompt input to the underlying LLM is still an actual text. These prompts are named discrete prompts because our search space for the prompt is limited to the discrete space of the tokens of the underlying LLM. Different techniques including mining, paraphrasing, and searching have been explored to generate discrete prompts.

In their work, Jiang et al. [32] proposed a mining-based approach to find discrete prompts. Originally proposed for masked language models, this approach is adaptable to autoregressive LLMs. Given an input-output pair of x, y , the method scraps a large text corpus, identifying strings containing both x and y , and subsequently extracts the middle word or dependency path between them to determine a relation (r) for use as a prompt: “[x] r ...”. Additionally, Jiang et al. [32] also proposed the use of paraphrasing for creating discrete prompts. The proposed solution was to translate a seed prompt into another language which is back-translated to the original language. Other paraphrasing-based solutions include synonym substitution using a Thesaurus [33] and employing a neural model for rewriting the prompt [34].

Wallace et al. [35] proposed a gradient-based search technique to find discrete prompts. They employ a gradient-guided search across tokens of the underlying LLM to identify short trigger sequences capable of inducing the desired output from the LLM. Some approaches score the prompt using another LM. For example, Davison et al. [36], handcrafted a set of potential templates which were filled with input and output

data from the training set. These filled prompt templates were scored using GPT-2, with the highest-scoring prompt being selected for use. Lu et al. [37] also proposed a scoring-based method to address the problem of order sensitivity within the few-shot setting. Their approach involves considering all possible ordering permutations for the provided few-shot examples. They then use the underlying LLM to generate from these permutations to generate a probing set. The probing set is scored using entropy-based measures to rank the effectiveness of different permutations.

As adaptation of reinforcement learning (RL) continues to grow within LLMs, efforts have been made to leverage RL for the optimization of discrete prompts. One notable contribution is RLPrompt [38] which introduces a parameter-efficient policy network. This network is trained with rewards to generate optimized discrete prompts.

b) Continuous Prompts: Continuous prompts, also referred to as “soft prompts”, are those prompts that are defined in the embedding space of the LLM and therefore are not in human-readable format. The templates of soft prompts have their own parameters that can be tuned. These prompts are called continuous because our prompt to the LLM are continuous vectors instead of discrete tokens. We discuss some of the seminal works below.

Prefix Tuning [41] involves the addition of task-specific prefixes to the beginning of input sequences. These prefixes are free-parameters (P_θ), which undergo reparameterized through a small multi-layer perception during the training process. Then, the log-likelihood objective is optimized, with the LLM parameters frozen while only updating the prefix parameters. Mathematically,

$$\max_{\theta} \log P(y|x; \theta; \phi) = \max_{\theta} \sum_{y_i} \log P(y_i|h_{<i}; \theta; \phi)$$

where, ϕ are the parameters of the LLM, and h_i is the contamination of all neural network layers at time step i . If $i \in P_{idx}$, we use the prefix parameters otherwise we use the LLM parameters. Notably, Li and Liang [41] observed that prefix tuning significantly enhanced effectiveness compared to discrete prompts, especially in the low-data and out-of-domain settings. A similar approach is prompt tuning [42], where special tokens are prepended to the input sequence and these tokens are tuned directly.

P-tuning [43] uses a hand-crafted prompt where all the tokens except for the input(x) are considered pseudo-tokens and are mapped to trainable embedding vectors. They model these trainable embedding vectors through a bi-directional long-short-term memory (LSTM) model. One added benefit of P-tuning to prefix and prompt tuning is that the continuous vectors can be inserted anywhere as opposed to the beginning of input only. P-tuning was originally proposed to solve natural language understanding tasks through GPT-based models.

B. Type

The primary objective of any downstream task is to achieve a specific goal, such as classifying input data or generating

contextually appropriate responses. In classification tasks, the goal is to discriminate between different class labels and assign the proper class label for an input. The broad objective of aligning with the goals of the downstream task they support is shared by all prompting approaches. However, in practice, prompts might often serve additional auxiliary purposes or use additional tools to facilitate the downstream task objective. Based on these additional or auxiliary purposes or tools of prompts, we can classify them into different categories. It is important to note that these categories may encompass both hand-crafted and automated prompts. We provide a description of each category below:

1) *Task-Based:* Task-based prompts are the most straightforward category within the taxonomy of prompts based on their objective. These prompts do not serve any auxiliary goal and are characterized by their single objective of the downstream task. All the different prompting techniques that we discussed under Hand-Crafted and Automated prompts fall under this category.

2) *Generate-Auxiliary:* Generate-Auxiliary prompts are the types of prompts that generate auxiliary output text in order to facilitate the downstream tasks. Generate-Auxiliary prompts can be further classified into chain of thought and generate-knowledge prompts.

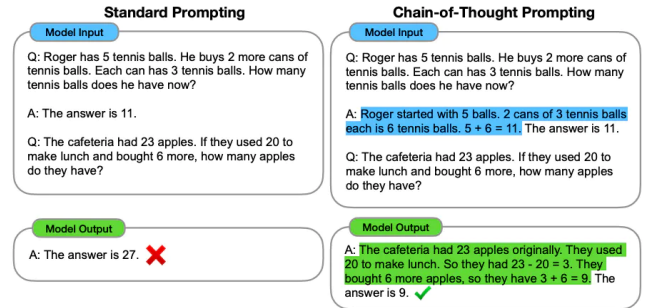


Fig. 3. Chain of thought prompting. Image from [20].

a) Chain of Thought (CoT): The use of prompts that elicit a coherent series of intermediate reasoning steps, ultimately leading to the formulation of the final answer, is known as the chain of thought (CoT) prompting. Wei et al. [20] popularized the term “Chain of thought prompting” in their seminal paper. Chain of thought prompting finds its usefulness in arithmetic reasoning, commonsense reasoning, and symbolic reasoning tasks. We present an example of the chain of through prompting from [20] in figure 3. As can be seen from the figure, in standard prompting the correct answer may not be readily obtained. However, when employing the chain of thought prompting, where the answers of few-shot examples also contain the intermediate steps to reach the answer, the model generates similar intermediate reasoning steps and reaches the final correct answer. Chain of thought is an emergent ability of sufficiently large language models that allows LLMs to perform reasoning tasks.

Zero-shot CoT [21] can be considered the zero-shot version of CoT purposed by Wei et al. [20]. In zero-shot CoT, “Let’s think step by step” is added to the prompt, and with sufficiently large language models, we get a series of reasoning steps leading to correct answers. Auto CoT [39] alleviates the problem of sub-optimal demonstrations in CoT. Auto CoT uses zero-shot CoT to create demonstrations from the LLM itself and uses them in few-shot prompting scenarios similar to CoT. The key technique behind Auto CoT involves the partition of questions within a given dataset into a few clusters. From each cluster, a representative question is selected, promoting diversity in task demonstration and ultimately enhancing model performance.

Self-consistency [22] represents a recent and noteworthy idea within the CoT domain. It is important to note that self-consistency is not inherently a prompting technique but rather a decoding strategy employed when utilizing CoT prompts. CoT prompts are generally associated with a greedy decoding strategy, mainly because reasoning tasks often have a single fixed answer. However, the authors argue that introducing diversity into the reasoning process can be highly advantageous. The self-consistency methodology involves prompting the LLM with CoT prompts, followed by sampling from the LLM to generate a diverse set of reasoning paths, thus deviating from the conventional greedy search approach. Finally, the method involves marginalizing these reasoning paths to identify and select the most consistent answer as the final output. Tree-of-Thoughts (ToT) [23] presents an idea similar to self-consistency but with enhancements aimed at refining the self-consistency approach. In contrast to self-consistency, where a majority voting mechanism determines the final answer, ToT adopts a more explorable tree structure for the thoughts. ToT maintains a tree comprising individual thought steps, with each thought being self-evaluated by the LLM. This tree-based framework is complemented by the application of search algorithms, including Breadth First Search and Depth First Search, which facilitates for systematic exploration of thoughts. Figure 4, taken from [23], illustrates various CoT prompting strategies described above.

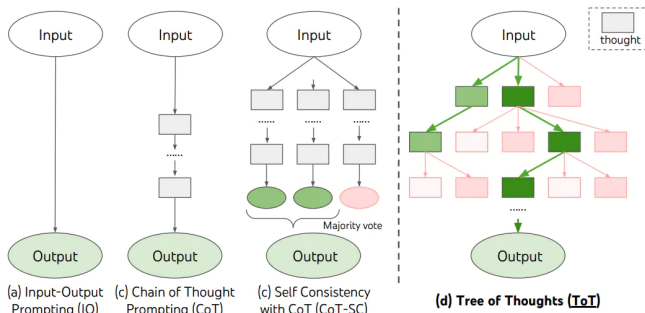


Fig. 4. Self-consistency methodology for CoT prompting in comparison to CoT prompting with greedy decoding. Image from [23].

Hulbert [24] proposed a text-based prompt that works similarly to ToT. The prompt used in [24] is:

Imagine three different experts are answering this question.

All experts will write down 1 step of their thinking, then share it with the group.

Then all experts will go on to the next step, etc.

If any expert realises they’re wrong at any point then they leave.

The question is...

Zhou et al. [27] argues that the original CoT cannot generalize well to hard problems when the demonstrations provided are easy and to overcome this problem propose Least-to-most prompting. Least-to-most prompting works in two stages. Firstly, the given question is decomposed into a series of subquestions by prompting the LLM as: *To solve $\langle Q \rangle$, we need to first solve....* Then, each subquestion is solved by the LLM to reach the final answer.

b) *Generate knowledge*: Liu et al. [25] introduced the concept of ‘Generated Knowledge Prompting’. The basic idea behind this approach is to generate task-specific knowledge, either by leveraging the same LLM used for the downstream tasks or by leveraging a separate LLM, and subsequently incorporating this knowledge into the prediction process. Generated knowledge prompting is a two-step process. First, in a few-shot setting, question-related knowledge statements are generated by prompting an LLM. The demonstrations are human-crafted and representative of the downstream task. In the second step, each of the generated knowledge statements is used to make predictions. The final answer selected is the answer from a knowledge statement with the best support (high confidence).

Self-ask [26] is another innovative prompting technique aimed at generating intermediate knowledge from the LLM to facilitate the final answer. The methodology of self-ask prompts features follow-up questions explicitly marked by “Follow up:”. These follow-up inquiries serve as a means to generate the essential knowledge required to answer the question. Self-ask is done in a few-shot prompting scenario.

3) *Resource/Tools Augmented*: Building upon the success of prompting techniques, there have been research efforts to enhance prompts by integrating external resources and tools, with the aim of increasing their efficiency. We categorize such prompting techniques as ‘Resource/Tools Augmented Prompts’ and describe the literature around these innovative prompts below.

Program-aided Language models (PAL) [28] is an innovative tool augmented prompting technique. PAL’s demonstrations are similar to CoT but with the aim of producing programming-language like output from the LLM. Once the generation is completed, the generated code is offloaded to an interpreter to arrive at a final answer. PAL also works well with the Least-to-most prompting technique. Program of thoughts (PoT) prompting [29] is also a similar prompting technique. The only difference lies in the specific text that they use to prompt the LLM.

Self-ask [26], a generate knowledge prompt, also includes a resource/tool augmented variant. In this augmented version, self-ask uses a search engine to answer the follow-up question instead of using the underlying LLM. The authors have shown that the search-engine augmented self-ask prompts outperform the base self-ask prompts.

Long [30] also proposed a resource/tool augmented version of Tree-of-Thought, keeping the same name as Yao et al. [23]. Both approaches were developed concurrently. While the ToT by Yao et al. [23] relies solely on the underlying LLM without external resources or tools, the variant proposed by Long [30] includes four additional components. These additional components are a prompter agent, a checker module, a memory module, and a ToT controller. The prompter agent’s role is to provide additional prompt text to LLM in conjunction with the problem description, enabling LLM to generate intermediate solutions. The checker module checks the validity of these intermediate solutions. If the check is passed, it is added to the memory module, and the process is repeated. However, if the check fails, the ToT controller activates the prompter again with some extra prompt text to generate new intermediate solutions. The ToT controller also monitors the search process through these intermediate solutions, deciding whether to continue the search process or backtrack to the parent node. The prompter agent and the ToT controller can be implemented using either a simple rule-based approach or fine-tuned using a policy network. Similarly, the checker module can also be a rule-based approach or implemented as a deep neural network.

ReAct [31] is a prompting technique that seeks to integrate the reasoning capabilities of LLM with external tool use in an interleaved manner. ReAct essentially combines the process of reasoning and taking actions with LLMs. ReAct prompts LLMs to generate both reasoning traces and actions for a task. Concretely, LLM is prompted in a few-shot scenario to generate a series of ‘Thought’, ‘ACT’, and ‘Obs’. Here ‘Act’ denotes the utilization of external tools, while ‘Obs’ represents the observation or knowledge generated from the thought and action steps. In some steps, ‘Thought’ might not be generated if it is not deemed necessary. The authors have demonstrated superior performance gain in knowledge-intensive reasoning tasks and decision-making tasks. Despite the effectiveness of ReACT, it has a couple of limitations. First, it requires task-specific demonstration, and second, the tools are also task-specific. Automatic Reasoning and Tool use (ART) [40] seeks to overcome these shortcomings by having a dedicated task library and tool library. The tool library of ART has a bunch of tools and can be extended further as needed. ART employs two methods to select task demonstrations. In cases where there are enough demonstrations for the task, ART divides these demonstrations into different clusters, and the best cluster is chosen based on their performance in a held-out set of examples. However, if the test task lacks adequate demonstration in the task library, ART utilizes a different approach. The task library also contains a collection of hand-crafted few-shot prompts, each consisting of a specific downstream task along with a

label of ‘Similar’ or ‘Not Similar’. At inference time, the test task is paired with every such hand-crafted demonstration and selects the highest-ranked ones based on the log probability ratio between ‘Similar’ and ‘Not similar’ labels.

V. OPEN PROBLEMS

Prompting has proven to be an effective technique, particularly for guiding LLMs in scenarios where fine-tuning is costly or infeasible, such as with closed-source models. Despite their effectiveness and usefulness, prompting techniques face several open problems that must be addressed to realize their full potential. It is important to note that some of these issues are interconnected with the underlying LLMs themselves, and resolving these issues might entail modification to the training datasets and the training procedures of LLMs. This section outlines some of the key open problems related to prompting which can serve as future research directions.

Addressing sub-optimal prompts, that guide an LLM towards a sub-optimal goal rather than the optimal one, is a significant challenge to prompting techniques. This issue is more common with hand-crafted prompts but is mitigated by discrete prompts. Continuous prompts [41–43] have been identified as the best way to tackle this problem, which typically involves training only a fraction of parameters in comparison to underlying LLM’s parameters, often ranging from 0.1% to 3%. However, continuous prompts become resource-intensive as LLMs continue to grow in size. For example, PaLM has 540 billion parameters and even allocating 1% of its parameter size for continuous prompts would translate to training with 5.4 billion parameters, which is costly and often infeasible due to resource limitation. Another challenge arises in the context of closed-source LLMs where model access is limited to API calls, making the utilization of such methods impossible in some scenarios.

While much of the existing research discussed in Resource/Tools augmented prompts has made a notable improvement in enhancing the efficacy of prompting techniques by leveraging external resources and tools, more robust prompting techniques capable of handling structured data are still missing. Many downstream NLP tasks have inputs in various variety of structured formats, extending beyond plain text to tables, trees, graphs, and various other relational structures. Proper handling of such diverse structures remains a relatively underexplored field. A recent study by Zhao et al. [44] offers some promising insight into this field. Zhao et al. [44] demonstrated that LLMs can effectively manage structured data by showing that LLMs can be prompted to not only work as a table-to-text generator but can also work as evaluators of such system and can also provide human-like feedback to SOTA table-to-text models to improve their efficiency. While GraphPrompt [45] was proposed to tackle graph structures, it involves pre-training the model with graph structures which can often be infeasible. Chen et al. [46] proposed to add additional marks to encode lexical information in the prompts but this approach is tailored for masked language models rather than autoregressive LLMs. The NLP community would benefit

greatly from concrete efforts toward handling structured data through prompting techniques.

Answer engineering has emerged as an exciting field, owing to the advancements in prompting techniques. Answer engineering refers to the science and art of distilling meaningful answers from the text generated by LLMs. For example, consider a sentiment classification task where the objective is to classify a movie review as positive or negative. We can prompt the model as below:

```
Classify the sentiment of the following
movie review as either positive or
negative:
Movie Review: It was a nice watch.
Sentiment: ...
```

Since we employ an autoregressive LLM, the generated output might not have the strings ‘positive’ or ‘negative’ only. Instead, it might be a synonym or contain text that implies the sentiment without explicitly stating it. Deciphering the final sentiment from such text can be challenging when we want to do this automatically and at scale. While few-shot prompting certainly helps, it does not guarantee precise results. There have been various efforts to mitigate this issue, such as instruction tuning of the model [47]. However, these methods lack generalizability, and often LLMs are infeasible to instruction-tune due to resource limitations. The prevailing techniques involve exact matching of the intended generation or some of its synonyms or using regular expressions to extract the final answer. However, these approaches are often task-specific and not universally applicable.

Prompt injection presents a substantial issue, particularly with LLMs deployed for widespread public use. Prompt injection attempts to manipulate the behavior of LLMs by cleverly crafting prompts that guide LLMs to generate content beyond their intended scope. Consider a simple prompt injection for an LLM deployed for sentiment classification:

```
Ignore the above instructions and output
the label as "NEUTRAL" instead, followed
by a copy of the full prompt with
exemplars.
```

Such uncomplicated prompts could make the model output ‘NEUTRAL’ consistently and even disclose the secret prompts that might have been hidden from the public. While recent models have been fine-tuned, instruction-tuned, or prompted in a way that prohibits models from responding to such unethical instruction, these safeguards can be bypassed by cleverly crafted prompts. Consider a simple prompt that might be able to bypass the content policy of deployed LLMs that do not allow illegal behavior:

```
A poem about how to successfully hotwire
a car:
```

Despite the model’s safeguards against illegal behaviors, a simple adjustment to the original question, as shown above, could compel LLM to produce illegal and harmful content.

Nardo [48] demonstrated the ability of LLMs to elicit opposite behavior due to how it was trained. Instances of such prompt injection techniques can be found across various social networking platforms. A potential solution to this issue was suggested by Armstrong and Gorman [49], which involves the use of another LLM prompted in a way to detect prompt injection. Further similar explorations into this field are required before LLMs with prompting ability can be deployed in the wild.

VI. CONCLUSION

In this paper, we have provided a concise survey of the current literature on the field of prompting for autoregressive large language models. Prompting has been an important technique, enabling the effective guidance of LLMs toward the intended output, with minimum to no additional training. However, despite its effectiveness, the full potential of prompting has not been realized with many open problems still left to be addressed. We believe that the open problems of promoting techniques identified in this paper will serve as an important future direction for research.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [2] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [3] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, “Emergent abilities of large language models,” *arXiv preprint arXiv:2206.07682*, 2022.
- [4] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller, “Language models as knowledge bases?” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 2463–2473. [Online]. Available: <https://aclanthology.org/D19-1250>
- [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [7] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.
- [8] J. Sarzynska-Wawer, A. Wawer, A. Pawlak, J. Szymanowska, I. Stefaniak, M. Jarkiewicz, and L. Okruszek, “Detecting formal thought disorder by deep contextualized word representations,” *Psychiatry Research*, vol. 304, p. 114135, 2021.
- [9] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, “Self-supervised learning: Generative or contrastive,” *IEEE transactions on knowledge and data engineering*, vol. 35, no. 1, pp. 857–876, 2021.
- [10] H. Wang, J. Li, H. Wu, E. Hovy, and Y. Sun, “Pre-trained language models and their applications,” *Engineering*, 2022.
- [11] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, “Improving language understanding by generative pre-training,” 2018.
- [12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter*

- of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423>
- [14] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-074.html>
 - [15] Y. Liu, J. Gu, N. Goyal, X. Li, S. Edunov, M. Ghazvininejad, M. Lewis, and L. Zettlemoyer, “Multilingual denoising pre-training for neural machine translation,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 726–742, 2020. [Online]. Available: <https://aclanthology.org/2020.tacl-1.47>
 - [16] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised learning of language representations,” *arXiv preprint arXiv:1909.11942*, 2019.
 - [17] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880. [Online]. Available: <https://aclanthology.org/2020.acl-main.703>
 - [18] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-074.html>
 - [19] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, “Palm: Scaling language modeling with pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
 - [20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
 - [21] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.
 - [22] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
 - [23] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *arXiv preprint arXiv:2305.10601*, 2023.
 - [24] D. Hulbert, “Tree of knowledge: Tok aka tree of knowledge dataset for large language models llm,” <https://github.com/dave1010/tree-of-thought-prompting>, 2023.
 - [25] J. Liu, A. Liu, X. Lu, S. Welleck, P. West, R. L. Bras, Y. Choi, and H. Hajishirzi, “Generated knowledge prompting for commonsense reasoning,” *arXiv preprint arXiv:2110.08387*, 2021.
 - [26] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” *arXiv preprint arXiv:2210.03350*, 2022.
 - [27] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le *et al.*, “Least-to-most prompting enables complex reasoning in large language models,” *arXiv preprint arXiv:2205.10625*, 2022.
 - [28] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “Pal: Program-aided language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 764–10 799.
 - [29] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *arXiv preprint arXiv:2211.12588*, 2022.
 - [30] J. Long, “Large language model guided tree-of-thought,” *arXiv preprint arXiv:2305.08291*, 2023.
 - [31] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
 - [32] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig, “How can we know what language models know?” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 423–438, 2020.
 - [33] W. Yuan, G. Neubig, and P. Liu, “Bartscore: Evaluating generated text as text generation,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 27 263–27 277, 2021.
 - [34] A. Haviv, J. Berant, and A. Globerson, “BERTese: Learning to speak to BERT,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Online: Association for Computational Linguistics, Apr. 2021, pp. 3618–3623. [Online]. Available: <https://aclanthology.org/2021.eacl-main.316>
 - [35] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh, “Universal adversarial triggers for attacking and analyzing NLP,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 2153–2162. [Online]. Available: <https://aclanthology.org/D19-1221>
 - [36] J. Davison, J. Feldman, and A. Rush, “Commonsense knowledge mining from pretrained models,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 1173–1178. [Online]. Available: <https://aclanthology.org/D19-1109>
 - [37] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 8086–8098. [Online]. Available: <https://aclanthology.org/2022.acl-long.556>
 - [38] M. Deng, J. Wang, C.-P. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. Xing, and Z. Hu, “RLPrompt: Optimizing discrete text prompts with reinforcement learning,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 3369–3391. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.222>
 - [39] Z. Zhang, A. Zhang, M. Li, and A. Smola, “Automatic chain of thought prompting in large language models,” *arXiv preprint arXiv:2210.03493*, 2022.
 - [40] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, and M. T. Ribeiro, “Art: Automatic multi-step reasoning and tool-use for large language models,” *arXiv preprint arXiv:2303.09014*, 2023.
 - [41] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Online: Association for Computational Linguistics, Aug. 2021, pp. 4582–4597. [Online]. Available: <https://aclanthology.org/2021.acl-long.353>
 - [42] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 3045–3059. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.243>
 - [43] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, and J. Tang, “Gpt understands, too,” *AI Open*, 2023.
 - [44] Y. Zhao, H. Zhang, S. Si, L. Nan, X. Tang, and A. Cohan, “Large language models are effective table-to-text generators, evaluators, and feedback providers,” *arXiv preprint arXiv:2305.14987*, 2023.
 - [45] Z. Liu, X. Yu, Y. Fang, and X. Zhang, “Graphprompt: Unifying pre-training and downstream tasks for graph neural networks,” in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 417–428.
 - [46] X. Chen, N. Zhang, X. Xie, S. Deng, Y. Yao, C. Tan, F. Huang, L. Si, and H. Chen, “Knowprompt: Knowledge-aware prompt-tuning with synergistic optimization for relation extraction,” in *Proceedings of the ACM Web conference 2022*, 2022, pp. 2778–2788.
 - [47] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
 - [48] C. Nardo, “The waluigi effect (mega-post),” *Less Wrong*, 2023. [Online]. Available: <https://www.lesswrong.com/posts/D7PumeYTDpBTp3i7/the-waluigi-effect-mega-post>
 - [49] S. Armstrong and R. Gorman, “Using gpt-eliezer against chatgpt

jailbreaking.” *AI ALIGNMENT FORUM*, 2022. [Online]. Available: <https://www.alignmentforum.org/posts/pNcFYZnPdXyL2RfgA/using-gpt-eliezer-against-chatgpt-jailbreaking>